🐳 Docker — Step 3

Docker Compose, Volumes, Live Reload & the Mental Model — A Beginner's Complete Guide

Project: `step3-compose` • A visit counter app (Node + Express) talking to Redis • Generated 25 June 2026

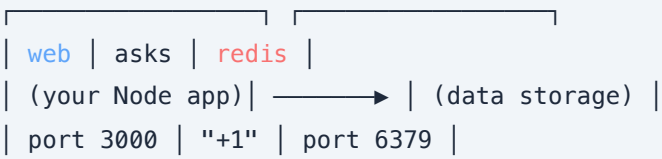
What this guide covers. Everything in your Step 3 project — both what the files already contain *and* the important concepts that aren't written in the files but you need to truly understand them: how Compose works, how volumes persist data, how live reload with nodemon works, how to get inside containers, how dependencies are installed the Docker way, and how all of this maps to professional practice.

📋 **Contents**

1. The Big Picture

Step 3 is about one powerful idea: **running multiple containers together that talk to each other.**

Your app is split into **two separate containers (called "services")**:



```
└──────────────────┘ └──────────────────┘

                  both on Docker's private network
```

The **web** service counts visits but doesn't store them itself. It hands the number to **redis**, which keeps it safe. That's why the count survives even when the app restarts — the data lives in a different container.

> 💡 **Why split them?** In the real world, your app and your database are separate, independently-scalable, independently-restartable things. Step 3 teaches that separation in miniature.

2. The Three Files

Your project is made of three files that work as a team:

File	Role	Analogy
`index.js`	The app's actual code	The meal you're cooking
`Dockerfile`	Recipe to build the app's *image*	The recipe card
`docker-compose.yml`	Runs & wires *multiple* containers	The head chef / conductor
`package.json`	Lists libraries & run scripts	The shopping list

📄 `index.js — the app (the most important line explained)`

```
const redisClient = createClient({
  url: process.env.REDIS_URL || "redis://redis:6379",
});
```

That middle word `redis` is **not** an IP address — it's the **service name** from `docker-compose.yml`. Docker's internal network turns that name into the correct address automatically. This is called *service discovery*.

```
const visits = await redisClient.incr("visits");  // increase counter, return new va
```

The app asks Redis to add 1 and return the new total. Because Redis (not the app) holds the number, restarting the app doesn't reset the count.

📄 `Dockerfile — the recipe for the web image`

```
FROM node:20-alpine          # start from a small Linux + Node base image
WORKDIR /app                 # work inside the /app folder

COPY package*.json ./        # copy ONLY dependency files first...
RUN npm install              # ...so this layer is cached when only code changes

COPY . .                     # now copy the rest of the source code
EXPOSE 3000                  # document that the app uses port 3000
CMD ["node","index.js"]      # the default command run when the container starts
```

> 💡 **Why copy `package.json` before the code?** Docker caches each step ("layer"). If you only change `index.js`, Docker reuses the cached `npm install` instead of re-downloading everything — much faster builds.

3. Docker Compose — the Conductor

`docker-compose.yml` describes *all* your containers in one file, then starts and connects them with a single command.

docker-compose.yml

```
services:
  web:
    build: .                        # build from THIS folder's Dockerfile
    ports:
      - "3000:3000"                 # Mac's 3000 → container's 3000
    environment:
      REDIS_URL: redis://redis:6379
    depends_on:
      - redis                       # start redis before web
    command: npm run dev            # dev override → runs nodemon
    volumes:
      - .:/app                      # live code: Mac folder → container
      - /app/node_modules           # keep container's node_modules separate

  redis:
    image: redis:7                  # ready-made image from Docker Hub
    volumes:
      - redis-data:/data            # persist Redis data in a named volume

volumes:
  redis-data:                       # declare the named volume
```

Key	What it does
`build: .`	Build this service's image from the local Dockerfile
`image: redis:7`	Don't build — pull a ready-made image from Docker Hub
`ports`	`"HOST:CONTAINER"` — exposes a container port to your Mac
`environment`	Sets environment variables inside the container
`depends_on`	Controls start order (redis before web)
`command`	Overrides the Dockerfile's `CMD` for this run
`volumes`	Connects storage (covered next)

> ⚠️ `depends_on` **caveat:** it waits for Redis to *start*, not to be fully *ready*. That's why `index.js` also connects to Redis before `app.listen()` — belt and braces.

4. Volumes — Persistence & Live Reload

A **volume** connects storage to a container. Your project uses volumes for *two different purposes*:

Purpose A — Live code reload (the `web` service)

```
volumes:
  - .:/app              # bind mount: your Mac folder ↔ container's /app
  - /app/node_modules   # anonymous volume: protect container's modules
```

- `.:/app` — a **bind mount**. Your Mac's current folder is mounted into the container, so when you edit a file on your Mac, it instantly changes inside the container too.
- `/app/node_modules` — an **anonymous volume**. Without it, the line above would overlay your Mac folder on top of `/app`, hiding the container's installed `node_modules`. This line "protects" the container's modules from being covered.

> 💡 **Two node_modules folders exist:** one on your Mac (for your editor's autocomplete) and one inside the container (what actually runs). They are separate by design.

Purpose B — Data persistence (the `redis` service)

```
volumes:
  - redis-data:/data    # named volume: survives container removal
```

This is a **named volume**. Redis writes its save file to `/data`, but that path is mapped to the `redis-data` volume, which lives *outside* the container. So destroying the container does **not** destroy the data.

Volume type	Syntax	Used for
Bind mount	`.:/app`	Live-editing your code
Anonymous	`/app/node_modules`	Protecting container-only folders
Named	`redis-data:/data`	Persisting important data

5. Nodemon — Automatic Restart

The bind mount makes your edited files appear inside the container — but plain `node` already started and won't re-read them. **Nodemon** watches the files and restarts the app on every change. That's what makes live reload actually work.

Nodemon needs to live in **two places**, each doing a different job:

Place	Job
`package.json`	Install it + define a `dev` script
The run command (`CMD` or compose `command`)	Actually run nodemon instead of `node`

package.json

```
"scripts": {
  "start": "node index.js",
  "dev":   "nodemon index.js"
},
"devDependencies": {
  "nodemon": "^3.1.0"
}
```

The pattern we chose: keep the `Dockerfile`'s `CMD` as `node index.js` (the clean "production" default), and override it for development in compose with `command: npm run dev`. This keeps one image usable for both dev and prod.

> 🚨 **The classic error:** `sh: nodemon: not found`. This means you added nodemon to `package.json` but ran `docker compose up` *without* `--build`, so the image was never rebuilt and nodemon was never installed. Fix: `docker compose down -v && docker compose up --build`.

6. Where Redis Data Actually Lives

Run `docker volume inspect step3-compose_redis-data` and you'll see:

```
"Mountpoint": "/var/lib/docker/volumes/step3-compose_redis-data/_data"
```

> ⚠️ **On a Mac, that path is NOT on your Mac.** Docker runs inside a hidden Linux VM, and that path lives *inside the VM*. You can't open it in Finder.

Three separate lifecycles explain why data survives:

Thing	Dies when…
Container	`docker compose down`
Files inside the container	container is removed
The named volume	**only** `docker volume rm` or `down -v`

> 💡 **Redis keeps data in RAM** and only writes the `dump.rdb` file to `/data` at save points, on graceful shutdown, or when you run `SAVE`. So an empty `/data` (`total 0`) just means it hasn't saved yet.

Compose prefixes volume names with the project (folder) name — that's why it's `step3-compose_redis-data`, not just `redis-data`. A different folder gets its own separate volume.

7. Getting Inside a Running Container

Two doors to the same room:

Command	Names the target by…	Works…
`docker compose exec redis sh`	**service name** (from yml)	inside a compose project folder
`docker exec -it step3-compose-redis-1 sh`	**real container name**	anywhere

The `-it` flags matter when you want to *stay inside*:

- `-i` = interactive (keep input open so you can type)
- `-t` = tty (give you a real terminal prompt)

```
# Run one command and exit:
docker compose exec redis ls -lh /data

# Stay inside and explore:
docker compose exec -it redis sh
```

> 💡 **Mental model:** service name → `docker compose exec`. Container name → `docker exec`. See both names with `docker compose ps`.

Reading `ls -lh`: `-l` = long format (permissions, size, date — one per line); `-h` = human-readable sizes (`1.0M` instead of `1048576`). Flags can stack: `-lh` = `-l -h`.

8. Installing Libraries the Docker Way

> 🚨 **Do NOT install libraries by hand inside a running container.** Anything you `npm install` inside a live container vanishes on the next rebuild, because containers are disposable.

The right workflow: change the recipe, then rebuild the image.

1. Get the package into `package.json` (the single source of truth):

```
# Either on your Mac:
npm install bcrypt
# ...or through the container:
docker compose exec web npm install bcrypt
```

2. Rebuild so it's baked into the image permanently:

```
docker compose down -v        # clear stale node_modules volume
docker compose up --build     # re-run npm install with the new package
```

The Dockerfile line that makes this work:

```
COPY package*.json ./
RUN npm install               # reinstalls everything in package.json on each build
```

> 💡 **The key shift in thinking:** stop thinking "install into the container." Start thinking "update `package.json` and rebuild the image." The build reads `package.json` — not your Mac's `node_modules`.

Goal	What to do
Add a library permanently	put it in `package.json` → `up --build`
Quick 5-minute experiment	`docker compose exec web npm install X` (knowing it vanishes)
Just look around inside	`docker compose exec -it web sh`

9. Essential Commands Cheat-Sheet

```
# ── Lifecycle ──────────────────────────────────────────
docker compose up                 # start everything (uses existing image)
docker compose up --build         # rebuild image, then start
docker compose up -d              # start in background (detached)
docker compose down               # stop & remove containers + network
docker compose down -v            # ...also remove volumes (wipes data!)
docker compose stop               # stop without removing
docker compose restart            # restart services

# ── Inspect ────────────────────────────────────────────
docker compose ps                 # list this project's containers
docker compose logs               # show all logs
docker compose logs -f web        # follow one service's logs live
docker volume ls                  # list all volumes
docker volume inspect NAME        # see where a volume lives

# ── Go inside / run commands ───────────────────────────
docker compose exec -it web sh    # open a shell in the web container
docker compose exec redis redis-cli   # open Redis CLI
docker compose exec redis ls -lh /data
```

10. Common Mistakes & Their Errors

You did this…	Error / symptom	Fix
Added a package but ran `up` without `--build`	`sh: nodemon: not found` / module not found	`down -v` then `up --build`
Ran `exec` with only a service name	`requires at least 2 arg(s)`	Add a command: `exec redis sh`
Expected `/data` to have a file immediately	`total 0` (empty)	Redis saves on intervals/shutdown; run `redis-cli save`
Looked for the volume path in Finder	Path doesn't exist on Mac	It's inside Docker's Linux VM — reach it via `exec`
Installed a library inside a live container	Gone after rebuild	Put it in `package.json` + rebuild

11. Is This the Professional Way?

The **core idea is genuinely professional** — but the dev setup has training wheels. Here's the honest split:

✅ What you're doing that IS how pros work

- `package.json` + `Dockerfile` as the single source of truth
- Rebuilding the image instead of hand-installing in containers
- Volumes for live reload + nodemon in development
- A named volume for data persistence

🔧 What changes in a real company

Topic	Professional practice
Installing deps	Commit `package-lock.json` and use `npm ci` in the Dockerfile for exact, reproducible installs
Images	Separate slim *production* image (multi-stage build, no dev tools) vs. the dev image with nodemon
Building	CI/CD (GitHub Actions, etc.) builds, tests, pushes to a registry, and deploys — humans rarely type `--build`
Data	`down -v` is dev-only; in production you never casually wipe data volumes (backups, managed DBs)
Databases	Redis/Postgres are usually managed services in prod, not in the same compose file

> 💡 **Bottom line:** What you've built is a legitimate professional *local-development* setup. The leap to production is mostly: lockfiles + `npm ci`, separate slim prod images, and CI/CD doing the building and deploying.

12. Glossary

Term	Meaning
Image	A frozen, read-only template built from a Dockerfile
Container	A running, disposable instance of an image
Service	A container defined in `docker-compose.yml`
Volume	Storage that lives independently of a container
Bind mount	A volume that maps a host folder into a container (for live code)
Named volume	Docker-managed storage with a name, used for persistence
Service discovery	Reaching another container by its service name as a hostname
Layer	A cached step in an image build
Registry	A place images are stored/shared (e.g. Docker Hub)
